Rules-Exact Conversational Tax Advisory: A Deterministic Engine with a Closed-Loop Acceptance Intelligence Layer

Jayant (12406581)
CSE472 : DEEP LEARNING FOR NATURAL LANGUAGE PROCESSING
Roll No. RK4P24MZA06
https://github.com/Jayant1711/Tax_Assist-system
November 2026

***Abstract*—**Tax numbers are easy to get wrong and hard for a reader to check, and language models get them wrong fluently. This report describes Tax Assist, a conversational advisor for Indian income tax that I built around a single rule: nothing outside the tax calculator is allowed to compute a rupee. The language layer only decides what the user probably meant. A versioned calculator prices the old and new tax regimes exactly, and every recommendation is ranked by re-running that calculator on a hypothetical top-up, so an "estimated savings" figure is really the calculator agreeing with itself twice. Two results from the work seem worth a reader's attention. I trained a neuro-fuzzy (TSK) head to imitate the calculator and it got worse every time I gave it more data; at 378k labeled rows its savings error was off by a factor of roughly fifteen thousand, and the reason is now clear, namely that the target contains a regime-flip $min()$ which additive rule bases smooth over instead of pinning. And since a confidence number nobody checks is just decoration, the system records what users actually do with each recommendation and computes Expected Calibration Error over those outcomes; a known-answer canary pins the instrument to zero, and a standing gate refuses any learned confidence until it measures ECE ≤ 0.05 on at least thirty resolved events. The shipped system passes 1,166 backend and 31 frontend tests, with extraction certified at macro-F1 1.000. The falsified experiment also reshaped the roadmap: the neuro-fuzzy layer is being repositioned not as a regression model but as an *explainable* rule base meant to capture how a human chartered accountant reasons about savings, which the exact calculator would then price.

***Index Terms*—**conversational AI, rule-based systems, neuro-fuzzy inference, confidence calibration, negative results, information extraction, tax computation

I. Introduction

Ask a chatbot about Section 80C of the Indian Income-tax Act and you will usually get a confident, well-formatted paragraph, sometimes even with the right numbers. The difficulty is telling when it didn't. Indian income tax is an unforgiving place to find out: liability comes out of two parallel regimes interacting with over twenty deduction sections, a senior-citizen matrix, surcharge with marginal relief, and a rebate clause, and the person reading the answer generally has no way to check any of it.

Early in this project a test profile earning Rs. 46 lakh produced a report in which every deduction saved exactly nothing. I spent the better part of a day convinced the calculator was broken. It wasn't. The new regime really does disallow those deductions, and my interface had simply failed to say so. That episode did more to shape the system than any design document I wrote, because it made the goal concrete: the fix was never "trust the model less," it was "show the working, from the calculator, every time."

Tax Assist is the system that came out of that. It is a web application in which a user describes their income and investments the way they would describe them to a human CA, in English or Hinglish, and gets back a full computation plus a ranked set of opportunities. The claim I defend is structural rather than statistical: no component of the system can invent a rupee. I enforce that in one place (the rules engine is the only calculator), verify it in another (recommendations are priced by re-running the engine), and watch it continuously (an append-only log and an event store that remembers what users did with the advice).

Six things in this report deserve mention as contributions. The two-pass pricing mechanism in Section IV makes every savings figure a recomputation rather than an estimate. The decision rules in Section V decide what to show and are exact by construction. Section V also contains what I consider the most useful result of the whole project, a falsification: the "obvious" learned head failed at three data scales, worse each time, and I can explain the mechanism precisely. Section VI describes the acceptance loop and the calibration gate. Section VII covers verification, where I want to flag one habit

that paid for itself repeatedly, which is treating a five-minute manual session in the running product as a first-class test. And Section VIII states where I intend to take the neuro-fuzzy layer next: away from regression, toward an explainable encoding of CA-style savings reasoning.

II. Background and System Overview

II-A. The tax domain, briefly

A reader unfamiliar with Indian income tax needs three facts. First, every taxpayer picks one of two regimes: the *old regime*, which allows deductions under numbered sections of the Act (80C for investments up to Rs. 1.5 lakh, 80D for health insurance, 80E for education-loan interest, 24(b) for home-loan interest, and so on) but taxes income at higher slab rates; and the *new regime* (Section 115BAC), which offers lower slab rates and a larger standard deduction but discards almost all of those deductions. Second, each deduction section has a statutory cap, so every head has a remaining *headroom* that a recommendation can fill. Third, the interaction is genuinely nonlinear: near certain income levels, topping up one deduction *flips* which regime is cheaper, so the saving from a top-up can be discontinuous in the input. Section V returns to this point, because it is exactly where a learned model of the savings fails.

II-B. The system

Tax Assist is a three-tier web application. The client is Next.js and React and presents a chat workspace plus the final advisory report. The backend is FastAPI with seventeen endpoints covering authentication, conversation, calculation, session management, recommendation events, and funnel analytics. Requests pass through wire-level validation, per-route rate limits, and an origin allow-list before anything touches application logic. A database holds users, server-side sessions, and the recommendation events; an append-only audit log records every interaction.

I fixed four decisions early and never revisited them, stated here plainly rather than by their internal names. The rules engine is the only calculator in the system; anything that needs a number asks it. The whole system runs offline and deterministically, so the same session state produces the same numbers on any machine. Extraction from free text stays rule-based rather than learned, a decision the evaluation numbers support. And every recommendation shown to a user is persisted as an event the user can later resolve with an action, which is what makes the feedback half of this paper possible.

III. Language Understanding

III-A. Processing chain

Every utterance goes through the same chain. Prepass guards mask or book percent signs, dates, and flat statutory amounts before any parsing happens. An Indian-numeral parser then handles digit forms (`1,25,000`, `18 LPA`, `45k pm`), word forms (“*thirty-five lakhs*”), and Hinglish numerals (“*paanch hazaar*”). A longest-match guard keeps statutory tokens from being misread; without it the section token `80u_80dd` decays into the amount “80” with a stray unit “u,” which turns a deduction section into eighty rupees of income. Finally, semantic attention decides which category an amount belongs to, and everything is clamped to a statutory range.

One correctness detail took me longer than it should have: clause splitting. A comma inside a digit run is grouping, not grammar, and a naive split on commas destroys `1,25,000` into three pieces. The production rule splits on a comma or semicolon only when neither neighbor is a digit, which as a regex is just two zero-width assertions:

```
(?<!\d)[,;](?!\d)
```
(1)

III-B. Semantic attention

Keywords vote for categories with a weight that decays by distance, so an amount binds to the statutory context nearest to it:

$$s(k) = 100 \cdot e^{-d_k^2/2\sigma^2}, \tag{2}$$

where d_k is the character distance from keyword k to the amount and $\sigma = 40$. In the sentence “invested 1.5 lakhs in PPF …pay 2L rent,” Section 80C wins the Rs. 1.5L at d=6 and the rent keyword wins the Rs. 2L at d=34, which is what a human reader would say too. When nothing scores above threshold the amount falls back to a profile default, flagged as provisional; a default can never overwrite a value the user actually booked.

III-C. Evaluation

I evaluate the language layer on template-disjoint splits of a 2,226-record labeled corpus, meaning test utterances never share a template with anything in training. Production extraction reaches macro precision, recall and F1 (the harmonic mean of the two, averaged over categories) of 1.000 across 21 categories on the 434-record evaluation split, abstentions included (17/17 correct), and clears the $\geq$ 95% certification gate on every income head and major section (Table I). The research layer’s intent classifier, a bag-of-words linear model over 8 intents, sits at 96.41%. A sequence-labeling network (BiLSTM) reached what the early evaluator reported as 0.143 entity accuracy — a number that later turned out not to be an accuracy at all: auditing the evaluator while expanding the template grammar showed the label encoding collapsed every entity tag to the outside class, the training loss diverged to NaN

TABLE I. Language-layer evaluation (template-disjoint splits).

Task	Model	Result
Extraction (21 cat.)	deterministic	**F1 1.000** (gate PASS)
Abstention	threshold	17/17 correct
Intent (8 classes)	linear classifier	96.41% acc.
Entities (5 types)	BiLSTM, 51 samples	0.143 acc. (off-path)

because every token was then ignored, and the reported figure was the softmax confidence of the resulting dead model, with per-class metrics synthesized from entity counts. After fixing the label encoding, building a deterministic corpus vocabulary, adding gradient clipping, and expanding the template grammar (six multi-entity sentence families), the same architecture measures 91.4% token accuracy with a macro-F1 of 0.756 across the six entity types. The corrected model remains research-track only — deterministic extraction still serves production — but the episode sharpened a policy this paper keeps returning to: an evaluator is code, and code can be quietly wrong in the direction of whatever number everyone already believed.

IV. The Deterministic Pricing Core

The calculator carries FY-2024–25 and FY-2025–26 as versioned rule registries: slab rates, surcharge with marginal relief, the health-and-education cess, the Section 87A rebate, the HRA exemption under its statutory least-of-three test, and per-section caps. Computation is a pure function of validated session state, so the same session yields the same numbers anywhere. The ML corpus used later in this paper is generated by this engine as well, from a fixed seed with LF-normalized writes and content-addressed manifests, which means a dataset can be rebuilt byte-for-byte if anyone doubts it.

IV-A. Two-pass recommendation pricing

For a candidate deduction head i with remaining headroom δ_i, the calculator prices the session as booked (pass one) and prices it again with the head topped up (pass two):

$$s_i \;=\; T_{\min}(x) - T_{\min}(x + \delta_i \cdot \mathbf{e}_i), \tag{3}$$

where $T_{\min}$ is whichever regime's total tax is cheaper. Since both passes are the authoritative calculator, s_i is not a guess about what the calculator would say. It is the calculator saying it twice.

IV-B. Exact per-deduction impact

The final report also prices each booked deduction by leaving it out and recomputing, in both regimes:

$$\Delta_i \;=\; T_r(x_{-i}) - T_r(x), \tag{4}$$

with r the recommended regime and x_{-i} the session with head i zeroed. The sign convention makes Δ_i positive exactly when the item delivered relief. I should admit that the first version of this equation subtracted the wrong way, and a Rs. 1,30,000 benefit showed up in the interface as negative. No amount of code review caught it; clicking through a live profile did. That experience is why manual end-to-end sessions count as tests in this project now. An item is labeled "applied" when the regime's taxable income actually changes without it, a recomputation test that correctly allows let-out home-loan interest under the new regime while still refusing self-occupied interest. In the live check, a profile with Rs. 5L of education-loan interest and Rs. 2L of let-out home-loan interest gives $\Delta_{80E} = Rs.\ 1{,}30{,}000$ and $\Delta_{24b} = Rs.\ 41{,}600$, both of which match a hand-worked slab calculation; a control profile under the new regime returns Rs. 0 for Section 80C and prints the statutory reason for it.

TABLE II. A learned neuro-fuzzy head across three data scales. More data made every gate worse; the exact rules pass all gates by construction.

Scale	Positives	Savings error	Opportunity
3.5k rows	146	fail	P 0 / R 0
77k rows	1,623	MAPE 795%	P 0 / R 0
378k rows	51,873	**MAPE 1,523,679%**	P 0 / R 0

V. Intelligence: Exact Rules and a Falsified Alternative

V-A. What the system decides, and how

Three plain questions decide what the final report shows the user: should this head be recommended at all, how much would filling it save, and how confidently should the system say so. Each question is answered from the recomputed quantities above, never from a model. Because the answers come from the same closed-form expressions the calculator uses, the rules measure zero error on a 10,995-row validation set: opportunity precision and recall of 1.0, savings error Rs. 0.00. That is not a lucky split. The rules cannot be wrong unless the calculator itself is wrong.

V-B. The learned alternative, falsified at three scales

The obvious research step is to train a model to imitate those decisions, and I did, three times, each time with more data than the last (Table II). The model was a Takagi–Sugeno–Kang neuro-fuzzy network, the standard adaptive fuzzy-inference design [1], [2], trained on engine-generated labels. The result was failure in every configuration, getting reliably worse as data grew. At the final scale the savings error reached 1,523,679% mean-percentage error, and the opportunity head never produced a single useful prediction at any scale.

It took a while to accept that this was structural and not a tuning problem on my part. The savings label is exactly

linear in two ratios the calculator can produce itself; when I hand the model those ratios as features, validation error drops to about 1.6×10^{-14}, in other words the label is a formula. The catch sits one level down: the regime choice inside those ratios is a discontinuous $min()$, and an additive rule base interpolates across the flip instead of pinning it. The boundary signal gets diluted, which shows up as a ranking accuracy of 65.26% at the final scale against 100% for the exact rules. The opportunity label is even starker: a top-up helps exactly when the current regime ignores that deduction, a condition that holds without exception across all 377,753 rows, so there is a distribution to learn only if one believes a rule is a distribution. My conclusion, now written down as policy so I do not waste a future month rediscovering it, is that learned regression on closed-form targets is inadmissible no matter how much data arrives. The dataset itself, 21,964 scenarios and 51,873 positive rows at 13.73% density, stays around as a deterministic benchmark rather than a growth axis.

VI. Closing the Loop: Acceptance and Calibration

VI-A. Acceptance events

Every ranked recommendation shown at the final report is written to a store as a shown-event carrying the deduction head, the top-up, the exact savings, and the confidence. The user resolves it later with an action: accepted, rejected, ignored, viewed, or dismissed. A funnel per session aggregates totals and per-head adoption, which is what lets the system eventually learn which kinds of advice people actually take.

Two implementation choices turned out to matter more than they look. Event identifiers are derived deterministically from the session and the head, so the identifier the interface renders is the identifier in the store. My first version generated random identifiers per call, which meant every card click would have hit an event that did not exist; I found that by clicking, not by reading code. Second, the closing-turn intelligence lives in one shared code path used by both the natural conversational close and the "`/end`" override. The override originally returned early and skipped the intelligence entirely, so sessions closed that way recorded no events at all. Both mistakes are now pinned by regression tests. In the live end-to-end check, a session shows three recommendations (Sections 24B, 80C and 80G, each at confidence 0.85), the user accepts one, and the funnel reports 3 shown, 1 accepted, rate 0.333, with per-head exact savings.

VI-B. Calibration and the standing gate

The confidence the system serves today is a constant: 0.85 when the calculator verified the number, 0.30 for priced-no-benefit. A constant is defensible precisely because its meaning can be stated in one line. Whether a learned confidence could replace it is a question I chose to instrument rather than guess at. Expected Calibration Error over equal-width bins,

$$\mathrm{ECE} = \sum_{b=1}^{B} \frac{|B_b|}{N} \left|\mathrm{acc}(B_b) - \mathrm{conf}(B_b)\right|, \tag{5}$$

is computed over resolved acceptance events. The instrument itself is validated by a known-answer canary: I construct 200 pairs whose observed acceptance equals each bin's average confidence exactly, and the implementation must report ECE 0.0 on them and must flag the flipped outcome set as miscalibrated. It does both, but the first canary did not; with ten items per bin, Python's banker's rounding turned a perfect construction into a reported ECE of 0.05, which is the sort of bug that would have quietly discredited the whole instrument. Twenty items per bin makes every bin's positive count an exact integer and the artifact disappears. The standing gate is then simple: no learned confidence ships until it measures ECE ≤ 0.05 on at least 30 resolved events without hurting the acceptance rate. The store currently holds one resolved event, so the honest measurement today is "unmeasured," and the anchor stays.

TABLE III. Verification summary of the shipped system.

Suite	Result
Backend (pytest)	1,166 passed / 0 failed
Frontend (vitest + tsc)	31 passed / clean
End-to-end (Playwright)	7 flows green
Extraction gate	macro F1 1.000 (PASS)
Decision rules	P/R 1.0, savings MAE Rs. 0.00
Calibration canary	ECE 0.0 (framework exact)
CI	3 jobs green per push

VII. Verification

Table III is the verification picture. Backend tests cover the calculator against hand-computed statutory cases, the language chain including adversarial numbering and Hinglish, session ownership and rate limits, the event store and its action endpoints, and the impact calculation's sign and new-regime behavior, that last pair pinned after the live session caught the defects. Continuous integration runs the backend suite, the frontend suite, and scripted browser end-to-end flows on every push. Two tagged releases went out through CI-gated pull requests, and I re-ran the full suite on the merged result both times.

VIII. Discussion: What the Neuro-Fuzzy Layer Is For

I want to close the technical part by being precise about where a fuzzy system genuinely belongs in this design, because the failed experiment above is easy to misread as "neuro-fuzzy does not work here," and that is not the lesson I take.

A neuro-fuzzy inference system is, structurally, a set of readable rules. Each rule reads roughly like: *IF income is high AND 80C headroom is large AND the taxpayer is young, THEN an NPS top-up is a strong opportunity*. The fuzzy membership functions give the conditions their graded, human flavor — "high," "large," "near the rebate cliff" — and the rule base as a whole is inspectable: unlike a neural network's weights, a trained fuzzy system can be translated back into sentences a person can argue with. That is exactly the shape of how a chartered accountant reasons about savings. A good CA does not regress a number; they carry a set of graded, conditional judgments — who is this client, which levers are still open, what is the interaction with the regime choice — and combine them into a plan.

So the direction I am taking the neuro-fuzzy layer is not regression but *representation*. Concretely, three steps. First, make the rule base explainable: every rule that fires for a given client should be translatable into one plain sentence of CA-style reasoning, shown alongside the exact numbers. Second, validate the decoded rules against a practicing CA's stated reasoning on the same profiles, so the fuzzy layer is judged as a model of professional judgment, not just as a predictor. Third, once the rules are explainable and validated, let the layer propose *plans* rather than single top-ups — combinations like "add a Rs. 50k NPS contribution, move to the old regime, and claim the pending 80D headroom" — with the deterministic calculator pricing every proposed plan exactly and the calibration gate governing how confidently each step is phrased. The engine prices; the fuzzy layer learns, in readable form, to think like a CA; and nothing either of them says is trusted until the engine confirms the rupees. That division of labor is the honest lesson of the falsification, turned into a research program instead of a dead end.

IX. Limitations

Salaried profiles are where the advisor is deep; business and capital-gains flows exist but are thinner, and I would not trust the business paths for anything real yet. FY 2025–26 is the only certified year, and a new fiscal year means adding a rules registry and re-certifying, which the design anticipates but which has not been exercised. The calibration gate is built and proven as an instrument but currently uninformative, since one resolved event is not a sample. The explainable-CA direction in Section VIII is a design intent, not a result; nothing in this paper should be read as claiming the fuzzy layer currently reasons like a CA. Personalization from acceptance data is deliberately blocked until the store holds roughly fifty resolved events. And the system advises; it does not file anything with anyone.

X. Conclusion and Future Work

The lesson I would keep from this project is that a tax advisor can be exact, and that exactness is an architectural property rather than a model quality. One deterministic calculator, everything else subordinate, costs almost nothing in capability, and it converts a trust problem into an engineering one. Every figure is recomputable, every claim ships with its evidence, and every failed idea is recorded alongside the measurement that killed it. If the project has a single most valuable machine-learning artifact, it is a negative result with the mechanism written down; its most valuable gate is one that keeps a model out until the data exists to judge it.

What comes next follows the evidence. An effectiveness dashboard over the acceptance funnel, with the Accepted-Rupee metric as the headline. One-click application of a recommendation through the calculator's own write path. And the neuro-fuzzy layer rebuilt toward the goal it was always meant for: an explainable rule base that captures how a chartered accountant thinks about legal savings, validated against a practitioner's judgment, proposing full savings plans that the exact calculator prices line by line. A FY 2026–27 rules drop will be the first real test of the registry's forward compatibility.

References

[1] T. Takagi and M. Sugeno, "Fuzzy identification of systems and its applications to modeling and control," *IEEE Trans. Syst., Man, Cybern.*, vol. SMC-15, no. 1, pp. 116–132, 1985.

[2] J.-S. R. Jang, "ANFIS: Adaptive-network-based fuzzy inference system," *IEEE Trans. Syst., Man, Cybern.*, vol. 23, no. 3, pp. 665–685, 1993.

[3] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, "On calibration of modern neural networks," in *Proc. 34th Int. Conf. Machine Learning (ICML)*, 2017, pp. 1321–1330.

[4] L. Chiticariu, Y. Li, and F. R. Reiss, "Rule-based information extraction revisited," in *Proc. 51st Annu. Meeting Assoc. Computational Linguistics (ACL)*, 2013, pp. 406–416.

[5] D. Bahdanau, K. Cho, and Y. Bengio, "Neural machine translation by jointly learning to align and translate," in *Proc. 3rd Int. Conf. Learning Representations (ICLR)*, 2015.

[6] M.-T. Luong, H. Pham, and C. D. Manning, "Effective approaches to attention-based neural machine translation," in *Proc. EMNLP*, 2015, pp. 1412–1421.

[7] I. Sutskever, O. Vinyals, and Q. V. Le, "Sequence to sequence learning with neural networks," in *Proc. NeurIPS*, 2014, pp. 3104–3112.

[8] T. Mikolov, K. Chen, G. Corrado, and J. Dean, "Efficient estimation of word representations in vector space," in *Proc. ICLR Workshop*, 2013.

[9] J. Pennington, R. Socher, and C. D. Manning, "GloVe: Global vectors for word representation," in *Proc. EMNLP*, 2014, pp. 1532–1543.

[10] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. NAACL-HLT*, 2019, pp. 4171–4186.

[11] A. Vaswani *et al.*, "Attention is all you need," in *Proc. NeurIPS*, 2017, pp. 5998–6008.

[12] *Income-tax Act, 1961* (India), §115BAC, inserted by the Finance Act, 2020; scheme of taxation under the new regime.